

MOOR

A Desired-State Control Plane for Docker Environments

Moor applies the reconciliation pattern that made Kubernetes reliable to the compose stacks most teams actually run: it continuously compares the state you declare with the containers you run, classifies every divergence as drift or intended change, alerts in Slack format, and repairs drift in seconds — with a complete audit trail.

Moor Platform Team

Infrastructure Reliability Engineering

Table of Contents

1. Executive Summary	1
2. The Problem: Configuration Drift	2
3. Market and Competitive Landscape	3
4. Goals and Non-Goals	4
Goals	4
Non-Goals	4
5. Personas and Use Cases	5
Maya — Platform Engineer	5
Daniel — Site Reliability Engineer	5
Priya — Application Developer	5
Primary Use Cases	5
6. Solution Overview: The Moor Control Plane	6
6.1 The Control Loop	6
6.2 Drift Versus Intended Change	7
6.3 Safety Model	7
6.4 Integration Surface	7
7. Functional Requirements	8
8. Non-Functional Requirements	9
9. Success Metrics	10
10. User Experience: CLI and Dashboard	10
11. End-to-End Demo Plan	11
12. Roadmap	12
13. Risks, Assumptions and Open Questions	13
References	14

1. Executive Summary

Every infrastructure team declares what its environments should look like in version-controlled files, and then watches reality diverge from that declaration, one hotfix, one console edit and one forgotten rollout at a time. The industry calls this **configuration drift**, and it is the quiet root cause behind a disproportionate share of production incidents. Uptime Institute reports that nearly 40% of organizations have suffered a major outage caused by human error over the past three years, and that 85% of those incidents stem from staff failing to follow procedures or from incorrect procedures themselves [1]. Gartner research projects that roughly 80% of outages impacting mission-critical services will be caused by people and process issues rather than technology [5]. When those mistakes land, they are expensive: over 90% of mid-size and large enterprises put the cost of a single hour of downtime above \$300,000, and 41% report costs between \$1M and \$5M or more [2].

+40%

of organizations have suffered a major outage caused by human error in the past three years (Uptime Institute, 2025)

80%

of mission-critical service outages are projected to come from people and process issues, not technology (Gartner)

\$300K+

is the cost of one hour of downtime for over 90% of mid-size and large enterprises (ITIC, 2024)

Moor is a desired-state control plane for Docker environments. It applies the reconciliation pattern that made Kubernetes reliable — an observe / diff / plan / act control loop — to the technology that the majority of developers actually touch every day. Stack Overflow's 2024 survey found Docker is the single most-used tool among professional developers at 59% adoption [4], and docker-compose remains the lingua franca of development, CI, and a long tail of staging and edge-production deployments. Yet compose stacks have no controller: nothing continuously compares what the compose file declares against what the Docker Engine is actually running, and nothing repairs the gap when it opens. Moor closes that loop. It watches the Docker Engine API, diffs live containers against the declared compose state, classifies every divergence as drift or intended change, alerts in Slack-compatible webhooks, and — in auto mode — remediates within seconds while writing an immutable audit trail.

This document specifies the problem, the product, and the acceptance bar for version 1.0, and it is paired with a fully working reference implementation: a portable docker-compose demo that runs in a GitHub Codespace or any machine with Docker, and proves detection, alerting, auto-remediation and intended-change convergence end-to-end with zero mock components. The demo is the PRD's executable appendix; every requirement marked P0 in Section 7 is implemented and observable in it.

2. The Problem: Configuration Drift

Configuration drift is the accumulated difference between the declared state of an environment — what the infrastructure code says should be running — and its actual state, which is what the Docker Engine, the cloud API, or the virtual machine is really doing at this moment [7]. Drift begins innocently: an engineer scales a cache manually during a traffic spike and forgets to revert it; a debugging session exports a stray environment variable into a production container; an urgent CVE fix is applied directly in the console because the pipeline is slow; a rollout dies halfway through and leaves half the fleet on the old image tag. Each change is individually rational and collectively corrosive. Within weeks, the environment that the team believes it is running and the environment that actually exists are different systems, and only one of them has ever been tested.

The consequences are not hypothetical. Uptime Institute's 2025 outage analysis found that nearly 40% of organizations suffered a major outage caused by human error in the last three years, with 85% of those incidents tracing back to procedural failures rather than lack of skill; 57% of respondents said their most recent major outage cost more than \$100,000 [1]. Gartner's core research has long projected that around 80% of mission-critical outages are caused by people and process issues [5], and its analysts estimate that more than 60% of enterprise network changes are still made manually [6]. IBM and Octopus Deploy both rank unmanaged drift among the top causes of app failure, prolonged development cycles, increased ticket volume, security exposure and compliance gaps [7][8]. The financial ceiling is set by ITIC's 2024 survey: for more than 90% of mid-size and large enterprises, a single hour of downtime now exceeds \$300,000, with 41% reporting \$1M to \$5M+ per hour [2].

Drift is especially dangerous because it attacks the assumption underneath every other reliability practice. Backups that quietly diverged from primary systems fail exactly when they are needed. A failover replica whose environment variables were hand-edited does not behave like the node it replaces. A staging environment that accumulated six months of untracked changes stops predicting production, so the bugs it should have caught — the 'works on my machine' class — surface directly in front of customers. The 12-factor methodology named dev/prod parity a first-class principle for precisely this reason: the gap between environments is itself a defect, and it compounds silently.

The industry's best answer to drift is the reconciliation control loop: declare state once in version control, and let a controller continuously force reality to match it. That pattern is the core of Kubernetes, whose controllers and operators reconcile cluster state every few seconds. But Kubernetes is a heavy prescription, and the 2024 DORA report confirms that most organizations still operate a broad mix of infrastructure flexibility levels, with platform approaches maturing unevenly [3]. A very large population of teams — the same population that made Docker the most-used developer tool at 59% [4] — runs docker-compose in development, CI, staging, and edge production, where no such controller exists. Terraform-style tooling detects drift only on the cloud-resource plane, at plan time, not at the runtime plane where containers actually run; commercial drift products (Spacelift, env0, Terraform Cloud) target the same cloud-IaC niche; and the compose ecosystem offers code-reloading (compose watch) but

nothing that enforces state. The result is a structural gap: the environments where drift is most likely to be introduced by hand are exactly the environments with no automated defense.

Servers do not fail because engineers are careless. They fail because the state everyone believes in and the state that actually exists quietly stop being the same thing — and nothing in the toolchain ever notices.

3. Market and Competitive Landscape

Drift management today clusters into four camps. Infrastructure-as-code platforms (Terraform Cloud, Spacelift, env0, Scalr) detect drift by scheduling terraform plans against cloud provider APIs — powerful for cloud resources, blind to runtime container state, and priced for platform teams rather than the compose long tail [9]. Kubernetes provides genuine runtime reconciliation, but adopting it to fix compose drift is a cure that costs more than the disease for most small stacks. Agentless OSS scanners such as driftctl popularized cloud-side drift listing but the project has wound down, leaving maintenance to forks. And the compose toolchain itself (compose watch, compose ps) reloads code and reports status, but performs no state enforcement. The following matrix summarizes the gap Moor occupies:

Approach	Runtime reconciliation	Compose-native	Operational weight	Primary gap
Terraform + drift scheduling (Spacelift, env0, TF Cloud)	No — plan-time only	No	Medium	Compares cloud APIs to IaC; never inspects running containers
Kubernetes controllers / operators	Yes	No	Very high	Requires cluster migration; overkill for single-host compose stacks
OSS drift scanners (driftctl lineage)	No — snapshot diff	No	Low	Cloud-resource listing only; project no longer actively maintained
Docker Compose + compose watch	No — code reload	Yes	Low	Rebuilds on file change; does not detect or repair state drift
Moor (proposed)	Yes — continuous	Yes	Low	Closes the loop for the compose ecosystem where most drift is introduced

Table 1: Drift-management approaches and the compose reconciliation gap

Moor's deliberate wedge is the intersection that no incumbent occupies: Kubernetes-grade reconciliation semantics delivered as a single container that speaks compose natively. This is not a fight for the Kubernetes market; it is a new floor for the population that Kubernetes will not reach this decade. The

near-term total addressable market is every team running compose in CI, staging, demos, appliances, and edge production — the same population the DORA 2024 data describes as operating at mixed infrastructure flexibility levels [3]. The long-term expansion path (Section 12) moves up-stack toward git-native desired state, multi-host agents, and policy guardrails, where Moor graduates from a control loop into a lightweight internal platform.

4. Goals and Non-Goals

Version 1.0 is a scope discipline exercise: make one loop, for one runtime, impeccable. The goals below define what impeccable means for the first release, and the non-goals exist to protect the wedge from scope creep that would turn Moor into a shallow clone of Kubernetes or Terraform rather than a deep solution for compose.

Goals

- Continuously reconcile docker-compose stacks against the Docker Engine API with a detection latency under 10 seconds for any drift event.
- Classify divergences as external drift (reality changed) or intended change (declaration changed), and converge both safely without operator intervention in auto mode.
- Provide an immutable, queryable audit trail of every drift, decision, and remediation, exposed via REST, SSE, and CLI.
- Ship Slack-compatible webhook alerting on drift detection and resolution with zero external dependencies beyond the webhook URL.
- Run as a single compose service with a Docker socket mount — no agents, no cluster, no database migration steps beyond one managed Postgres container.
- Prove every P0 requirement in a Codespace-portable demo that a skeptical engineer can run in under ten minutes with three commands.

Non-Goals

- No Kubernetes support in v1.0 — the reconciliation semantics are designed to be portable to it later (Section 12), but nothing in v1.0 requires a cluster.
- No cloud infrastructure drift (AWS/Azure/GCP resources) — that is the Terraform ecosystem's territory and is deliberately out of scope.
- No secret management, image scanning, or vulnerability detection — Moor reconciles state; it does not audit content.
- No multi-host orchestration in v1.0 — one control plane per Docker host or per compose project.

- No forced migration of existing workflows — Moor must be adoptable by adding exactly one service to an existing compose file.

5. Personas and Use Cases

Moor's design conversations revolve around three personas who experience drift from different altitudes. They share one property: none of them wants another dashboard to watch, which is why every Moor capability is also available as a CLI verb and a webhook payload.

Maya — Platform Engineer

Maya owns the internal platform that forty product teams deploy onto as compose stacks. She is measured on stability and toil reduction, and her pain is the 3 a.m. pager when a hand-mutated environment variable in a staging stack breaks a customer demo at 9 a.m. For Maya, Moor is a control plane she adds once to the stack template: advise mode everywhere, auto mode in staging and demo environments. She treats the event store as her incident ledger — when something breaks, the first question is no longer 'what changed?', because the audit trail answers it with timestamps, container IDs, and diffs.

Daniel — Site Reliability Engineer

Daniel is on-call for environments where a rogue docker run during a debugging session quietly changed the replica count or image tag of a stateful service. His manual remediation ritual — docker ps, diff against the compose file, recreate by hand, write the postmortem note — takes 20 distracted minutes under incident pressure. Moor compresses that to a 30-second automated repair with a webhook card in the incident channel and a complete audit record. Daniel's acceptance test is simple: kill a workload container, and have it back — correctly configured — before he finishes reading the alert.

Priya — Application Developer

Priya develops in GitHub Codespaces and occasionally tests in a shared staging compose stack. Drift reaches her as mystery: the bug she cannot reproduce locally because her container carries an environment variable nobody remembers setting. She will never open Moor's dashboard for fun, but she benefits from every reconcile cycle, and when she does interact with Moor it is through two surfaces: the drift banner in her terminal from moor status, and the intended-change flow — she edits the compose file, pushes, and the environment converges without her asking anyone to re-deploy it.

Primary Use Cases

- UC-1 Hotfix drift: an operator edits a running container to work around an issue; Moor detects the divergence, alerts, and (auto mode) restores declared state while logging the full diff.

- UC-2 Rogue scaling: someone scales a service manually to absorb load; Moor flags the extra replicas and converges to the declared count, preventing silent cost and configuration divergence.
- UC-3 Environment mutation: an injected or accidental env var reaches a container; Moor identifies the exact changed key and recreates the container with declared environment.
- UC-4 Intended change: an engineer edits the compose file (image bump, replica change); Moor classifies it as an intended change and rolls it out to the live environment — GitOps for compose.
- UC-5 Outage forensics: after an incident, the team queries the event store to reconstruct exactly when the environment stopped matching its declaration, and what Moor did about it.

6. Solution Overview: The Moor Control Plane

Moor is a small control plane built on one proven idea: a reconciliation loop. Kubernetes popularized the pattern — observe actual state, compare against desired state, compute a plan, act, repeat — and Moor applies it to the Docker Engine. The product ships as one container that mounts two things: the Docker socket, which is the source of actual state, and the compose file, which is the source of declared state. Everything else — the dashboard, the CLI, the alerting, the audit trail — reads from the loop's outputs. The name is the strategy: to moor a vessel is to hold it fast against drift.

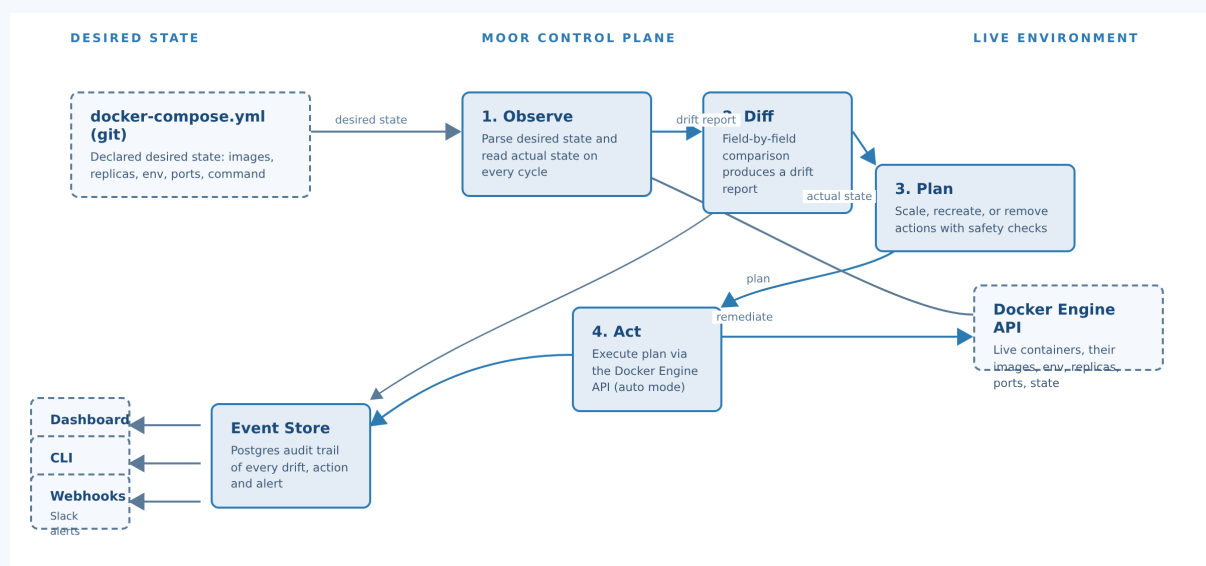


Figure 1: The Moor reconciliation control loop

6.1 The Control Loop

Every cycle executes four phases. **Observe** parses the compose file into a desired-state model — services with image, replica count, environment, command, and published ports — and queries the Docker Engine API for the actual state of every container carrying the compose project's labels. **Diff** compares the two models field by field and produces a structured drift report: missing replicas, extra replicas, image

mismatches, environment variable deltas, command and port divergence. **Plan** turns the report into ordered, idempotent actions — remove extras, recreate misconfigured containers, create missing ones — subject to safety rules. **Act** executes the plan through the Docker API and records the outcome. In advise mode the loop stops after Diff and raises the alarm; in auto mode it proceeds to Act. The cycle is deliberately boring: stateless, crash-safe, and correct regardless of how many cycles were skipped.

6.2 Drift Versus Intended Change

Blind enforcement would be dangerous, so the loop distinguishes who moved first. Moor retains the hash of the last-seen desired and actual states. When a divergence appears and the desired hash changed while actual state matches the previous desired snapshot, the event is classified as an **intended change** — the declaration moved, reality has not caught up, and the loop converges the environment forward. When the desired hash is unchanged and reality has moved, it is **drift** — someone or something altered the running environment, and the loop restores the declaration. This single classification turns Moor from a dangerous auto-reverter into a GitOps-style pull deployer, and it is what makes the same binary useful to both Maya and Priya.

6.3 Safety Model

Moor's blast radius is deliberately small. It only touches containers created by the compose project it monitors, identified by the standard `com.docker.compose.project` label — never foreign containers on the same host. Services can opt out of management with a single label, and the control plane excludes its own database, alert sink, and itself by default so the loop can never kill the components it depends on. All remediation actions are idempotent and rate-limited; a flapping container that fails to converge triggers back-off and an alert rather than a restart storm. Moor never executes user-supplied code, never mounts anything except the socket and a read-only compose file, and writes nothing to the host filesystem.

6.4 Integration Surface

The control plane exposes a REST API (state, drift, events, mode, reconcile triggers) consumed by three first-party clients: a server-sent-events dashboard for live observation, a typer-based CLI (status, plan, apply, watch, mode) that mirrors the terraform vocabulary engineers already know, and webhook alerting that emits Slack-compatible card payloads — the format every chat platform and incident tool already ingests. All events persist to a dedicated Postgres event store, giving teams a durable, queryable audit trail that doubles as incident-forensics input. The event schema is versioned from day one because it is the product's contract with its own ecosystem.

7. Functional Requirements

Requirements are grouped by release priority. P0 defines the v1.0 bar and is fully implemented in the reference demo accompanying this document; P1 is the first post-launch train; P2 defines the platform trajectory. Each requirement is written to be verifiable by observation of the running system, not by code inspection.

ID	Requirement	Pri
FR-01	Parse a docker-compose file into a desired-state model covering image, replica count, environment, command, and published ports per managed service	P0
FR-02	Read actual state from the Docker Engine API for all containers carrying the target compose project labels	P0
FR-03	Compute a field-level drift report (missing, extra, changed) within 10 seconds of any state divergence	P0
FR-04	Classify each divergence as drift or intended change using desired/actual state hash transitions	P0
FR-05	Advise mode: emit drift-detected events and alerts without modifying the environment	P0
FR-06	Auto mode: execute idempotent remediation (remove extras, recreate mismatched, create missing) with correct compose labels and networks	P0
FR-07	Exclude control-plane services from management via labels; never touch containers of other projects	P0
FR-08	Persist every event (detection, classification, action, alert, mode change) to the Postgres event store with timestamps and full payload	P0
FR-09	Emit Slack-compatible webhook payloads on drift detection and resolution	P0
FR-10	Expose REST endpoints for state, drift, events, mode, and manual reconcile trigger	P0
FR-11	Stream events to the dashboard over SSE with sub-second fan-out	P0
FR-12	Provide CLI verbs: status, plan, apply, watch, mode — with terraform-style diff output	P0
FR-13	Re-parse the compose file on mtime change and treat changed declarations as intended change	P0
FR-14	Render a live dashboard: service compliance cards, drift timeline, action audit, mode switch	P0
FR-15	Rate-limit remediation with back-off for non-converging services	P1
FR-16	Drift report API filtering by service, type, and time window	P1
FR-17	Dry-run mode for apply producing a plan without execution	P1
FR-18	Multiple compose projects per control plane with per-project policy	P1
FR-19	Policy rules: allowed drift windows, protected services, maintenance calendar	P2

ID	Requirement	Pri
FR-20	Git-native desired state: reconcile from a branch, not only a mounted file	P2

Table 2: Functional requirements by release priority

8. Non-Functional Requirements

Moor polices environments that are themselves part of the reliability path, so its own resource and failure envelope is part of the contract. The targets below are the acceptance bar for v1.0 and are measured in the reference demo environment (single host, 2 vCPU, 8 GB RAM — a standard Codespace).

Attribute	Target	Verification
Detection latency	Under 10 s from divergence to drift event, p95, at 10 s loop interval	Demo: injected drift surfaces in dashboard and webhook within 10 s
Reconcile cost	Full cycle under 500 ms for 50 containers on one host	Engine cycle timing instrumentation in event store
Control plane footprint	Under 200 MB resident, under 3% CPU at steady state	docker stats on moor container during demo
API responsiveness	p95 under 100 ms for state, drift and events reads	CLI/API smoke test with 100 sequential requests
Durability	No event loss on control-plane restart; loop resumes without manual action	Restart moor container mid-demo; verify event continuity
Portability	Runs on any Docker 24+ host: Codespace, Linux, macOS, CI runner	Demo executed from a clean Codespace with three commands
Security posture	Read-only compose mount; socket access only; no code execution; no egress beyond webhook URL	Compose file and Dockerfile review; network policy in v1.1
Operability	Single-service deployment; zero-migration Postgres schema via SQLAlchemy on startup	Fresh clone to running demo in under 10 minutes

Table 3: Non-functional requirements and verification methods

9. Success Metrics

Moor's success is measured on time, not opinions: how quickly drift is detected, how quickly it is repaired, and how reliably those two numbers hold. The v1.0 targets below are aggressive because the demo exists to prove them, and each metric is computed directly from the event store so that the product grades itself.

Metric	Definition	v1.0 Target
Detection SLO	Share of drift events detected within 10 s of divergence	100%
Auto-remediation MTTR	Median time from drift detection to declared state restored	Under 30 s
Alert delivery	Share of drift events producing a delivered webhook within 5 s	99%+
False positive rate	Share of drift reports that do not correspond to a real state change	Under 1%
Convergence correctness	Share of auto-mode repairs that restore exact declared state (field-level)	100%
Time-to-first-value	Clone to observing first drift event in the guided demo	Under 10 minutes

Table 4: Success metrics and v1.0 targets

Beyond the machine-facing numbers, the leading indicator of adoption is behavioral: the ratio of drift events resolved by Moor autonomously to drift events that page a human. Every point that ratio climbs is toil removed from Daniel's on-call rotation, and the event store makes the ratio a single SQL query. The lagging indicator is the incident review: six months after adoption, 'configuration drift' should stop appearing as a contributing factor in postmortems, because the window between divergence and repair will have collapsed from weeks to seconds.

10. User Experience: CLI and Dashboard

Moor presents one mental model through three surfaces, and the CLI is the canonical one because it is the surface engineers trust. The vocabulary is borrowed deliberately from terraform: status inspects, plan diffs, apply converges, watch follows. The output below is the actual transcript from the reference implementation after a drift injection — three rogue redis replicas and a mutated database environment variable — with auto mode armed:

```

$ moor plan

Moor — desired state review (project: moordemo)
Mode: AUTO Loop: 10s Compose: docker-compose.yml

web nginx:1.27-alpine replicas 1/1 env OK COMPLIANT
cache redis:7.2-alpine replicas 1/4 env OK DRIFTING
  ~ replicas: 1 declared, 4 running -> remove 3
db postgres:16-alpine replicas 1/1 env CHANGED DRIFTING
  ~ env POSTGRES_PASSWORD: declared ****, running ****
  -> recreate container with declared environment

Plan: 4 actions (2 services)
- remove container moordemo-cache-3 (extra replica)
- remove container moordemo-cache-2 (extra replica)
- remove container moordemo-cache-1 (extra replica)
- recreate container moordemo-db-1 (environment drift)

$ moor apply

drift.detected cache replicas 1 -> 4 t+0.4s
drift.detected db env POSTGRES_PASSWORD t+0.4s
alert.sent webhook slack-compatible card t+0.6s
action.ok removed moordemo-cache-3 t+1.2s
action.ok removed moordemo-cache-2 t+1.2s
action.ok removed moordemo-cache-1 t+1.3s
action.ok recreated moordemo-db-1 t+3.1s
drift.resolved cache, db t+3.4s
state 3/3 services compliant t+3.4s

```

The dashboard renders the same loop for humans who prefer light over terminals: compliance cards for every managed service (desired versus actual image, replica gauge, environment parity), a live event timeline fed by SSE that turns red on drift and green on resolution, an action audit trail with per-action outcome, and a mode switch with an explicit confirm step that makes the advise/auto boundary impossible to trip over. The design constraint for both surfaces is identical: every pixel is derived from real Docker Engine state — if the engine is lying, the UI is lying, and there are no synthetic indicators anywhere in the product.

11. End-to-End Demo Plan

The reference implementation is a self-contained repository that runs on any Docker 24+ host and is Codespace-ready via a devcontainer with Docker-in-Docker. The stack it manages is deliberately boring and real: an nginx web tier, a redis cache, and a postgres database, plus the Moor control plane, its event-store database, and a webhook alert sink. The demo is driven by a single narrated make target that executes the full problem-then-solution arc in under ten minutes:

- Act 1 — Baseline: docker compose up brings the stack to a compliant state; moor status shows 3/3 services compliant; the dashboard is green.
- Act 2 — The problem, observed: with Moor in advise mode, the drift injector kills the web container, scales cache to 4 replicas, and mutates the db environment. Within seconds the dashboard turns red,

the webhook sink receives a Slack-format alert card, and moor plan prints the exact drift — but nothing self-repairs, reproducing today's status quo of visibility without control.

- Act 3 — Control: the operator flips to auto mode. Moor removes the rogue replicas and recreates the mutated container; state returns to 3/3 compliant with every action in the audit trail; the alert sink records the resolution card.
- Act 4 — Resilience: drift is injected again; remediation now happens without human input, and the operator restarts the Moor container mid-cycle to demonstrate crash-safety and event continuity.
- Act 5 — GitOps: the compose file's web replica count is edited 1 -> 3. Moor classifies it as an intended change and converges the live environment forward — closing the loop in the opposite direction.

Every artifact in the demo is real infrastructure: real Docker API calls, real containers, a real Postgres event store, a real HTTP webhook receiver rendering the exact payload Slack would ingest. The injector is a legitimate chaos tool, not a simulation — it performs the same docker API mutations a tired engineer would. Acceptance for the demo is binary: a fresh Codespace, three commands (make setup, make demo, open the printed URLs), and all five acts must pass without manual intervention.

12. Roadmap

The roadmap graduates Moor from a compose control loop to a lightweight environment platform without ever abandoning the wedge. Each phase has a single exit criterion, and no phase begins until the previous one meets its metrics from Section 9 in the field.

Phase	Scope	Exit criterion
v1.0 (this PRD)	Single-project compose reconciliation, advise/auto modes, event store, CLI, SSE dashboard, webhooks	Demo passes end-to-end in a fresh Codespace; detection and MTTR SLOs met
v1.1 — Hardening	Back-off and flapping policies, drift report filters, dry-run apply, network policy, Helm-free single binary	30 days in production at 3 design partners with zero unhandled failures
v2.0 — Git-native	Desired state pulled from a git branch, PR-time plan previews, approval gates for auto mode, per-project policy	Intended changes flow exclusively through git for adopters
v3.0 — Multi-host	One control plane, many hosts via remote Docker contexts and edge agents; Kubernetes equivalence layer	Fleet of 10+ hosts reconciled from one Moor deployment

Table 5: Release roadmap and exit criteria

13. Risks, Assumptions and Open Questions

The honest risk register for a reconciliation product starts with the failure mode of its own success: a control loop that repairs the wrong thing is more dangerous than no loop at all. The dominant risks and their mitigations are listed below, followed by the assumptions this PRD depends on and the questions v1.1 must answer.

Risk	Impact	Mitigation
Auto mode fights a legitimate manual intervention during an incident	High	Adise-by-default; explicit mode switch; back-off on non-convergence; webhook announces every action it takes
Recreation of containers with stateful volumes loses data	High	Volume mounts are part of declared state and preserved on recreate; v1.1 adds a protected-service label that blocks recreation entirely
Compose spec interpretation drift across docker/compose versions	Medium	Desired-state model covers the stable core subset (image, replicas, env, command, ports); unsupported keys are ignored loudly, not silently
Socket access grants the control plane root-equivalent engine rights	Medium	Read-only compose mount, no code execution, no host writes; documented socket-group scoping; v1.1 ships a rootless proxy with narrowed API surface
Event store becomes a liability at high drift frequency	Low	Retention policy and table partitioning on the v1.1 train; event volume is bounded by action rate-limits

Table 6: Principal risks and mitigations

Three assumptions underpin the v1.0 scope: that teams will accept a single control plane per compose project in exchange for zero-friction adoption; that the Docker socket mount is an acceptable trust boundary for this persona (it is the same boundary every compose deployment already crosses); and that ten-second detection granularity is sufficient, since the manual alternative is measured in weeks. The open questions for v1.1 are the inverse of those assumptions: whether multi-project consolidation arrives organically from users or must be pushed, whether a narrowed engine proxy materially changes the security conversation for regulated adopters, and whether the git-native flow should standardize on a Moor-specific desired-state format or remain strictly compose-compatible. Each has an owner and a decision date in the platform review; none blocks v1.0.

References

- [1] Uptime Institute. (2025). Uptime announces annual outage analysis report 2025 [Press release].
<https://uptimeinstitute.com/about-ui/press-releases/uptime-announces-annual-outage-analysis-report-2025>
- [2] ITIC Corp. (2024). ITIC 2024 hourly cost of downtime report, Part 1.
<https://itic-corp.com/itic-2024-hourly-cost-of-downtime-report>
- [3] DORA. (2024). Accelerate state of DevOps report 2024. <https://dora.dev/research/2024/dora-report>
- [4] Stack Overflow. (2024). 2024 developer survey: Technology. <https://survey.stackoverflow.co/2024/technology>
- [5] OpsTrails. (2025). Why most production outages are self-inflicted.
<https://www.opstrails.dev/insights/80-percent-production-outages-self-inflicted>
- [6] NetBrain. (2025). How to stop network outages from hurting your business.
<https://www.netbrain.com/blog/stop-network-outages>
- [7] IBM. (2026). What is configuration drift? <https://www.ibm.com/think/topics/configuration-drift>
- [8] Octopus Deploy. (n.d.). Configuration drift: Causes, examples, risks, and best practices.
<https://octopus.com/devops/configuration-management/configuration-drift>
- [9] Snyk. (2022). Tools for infrastructure drift detection. <https://snyk.io/blog/tools-infrastructure-drift-detection>